설비 고장을 예측하는 AI 데이터 분석

Matplotlib 기초

선 그래프와 그래프 완성

posco

CONTENTS

목차

01 시각화 시작 — 왜 그래프로 보는가

02 선 그래프 그리기 — plot·옵션·여러 선

03 그래프 완성 — 제목·축·범례·저장

posco

posco

01

선 그래프와 그래프 완성

Matplotlib으로 첫 선 그래프부터 읽을 수 있는 완성형 그래프까지

posco

선 그래프와 그래프 완성

숫자표로 안 보이던 변화가 그래프 한 장에 드러남

이 모듈의 목표

누가 봐도 이해되는
읽을 수 있는 그래프 한 장 완성

선 그래프 그리기 → 제목·축·범례 붙이기 → 파일로 저장까지의 한 흐름

설비 데이터를 그림으로 봐야 하는 이유

사람 눈은 숫자 비교에 약하고 모양 인식에 강함

01 · 숫자표의 한계

숫자표는 흐름이 안 보임

수십 줄 쌓이면 개별 값만 보이고 전체 변화가 안 잡힘

그래프는 모양으로 보임

02 · 사람 눈의 특성

숫자 비교에 약함, 모양 인식에 강함

수백 숫자 동시 비교는 불가, 모양과 흐름은 즉시 알아챔

잘하는 일로 바꿔주는 도구

03 · 시각화의 역할

분석 도구이자 소통 도구

혼자 분석할 때도, 정비 담당자와 공유할 때도 같이 작동

예쁜 그림이 아닌 빠른 판독

선 30개 중 혼자 치솟는 선은 누구나 0.5초에 발견

posco

숫자표와 선 그래프의 차이

같은 7일치 온도, 보는 방식만 바꿔도 읽히는 정보가 달라짐

보는 방식	7일 온도	알 수 있는 것
숫자표	62 · 63 · 64 · 66 · 69 · 73 · 78	**개별 값만**
선 그래프	**오른쪽 위로 상승하는 선**	**추세 즉시**

온도가 계속 오르네, 문제 아닐까? 라는 판단이 그래프 한 장에서 즉시 가능

posco

matplotlib과 pyplot의 관계

큰 도구 상자 안의 자주 쓰는 공구함, 별명은 plt

01 · matplotlib

맷플롯립이라고 읽음

파이썬
그래프 도구 전체

그래프에 필요한 아주 많은 기능을 담은 큰 도구 상자

02 · pyplot

파이플롯, 별명 plt

자주 쓰는
도구 모음

선 그리기·제목 붙이기·화면 표시 명령이 모두 들어 있음

공구 창고(matplotlib) 안 자주 쓰는 공구함(pyplot)을 plt로 부르는 약속

posco

pyplot 불러오기와 별칭 plt

모든 그래프 코드의 시작점, plt는 전 세계 공통 약속

CODE

```
import matplotlib.pyplot as plt
# matplotlib 안의 pyplot을 plt로 부름
```

자주 하는 실수 3가지

가운데 점 빠뜨리기 · matplotlib 철자 틀리기 · as 빠뜨리기

numpy는 np, pandas는 pd, pyplot은 plt — 같은 방식의 별명 짓기

posco

그래프 그리는 기본 4단계 흐름

어떤 그래프든 준비 → 그리기 → 꾸미기 → 보여주기 순서로 흐름

STEP 1

준비

도구를 불러오고 그릴 데이터를 변수에 담기

`import · 데이터`

→

STEP 2

그리기

plot 같은 명령으로 데이터를 선으로 바꾸기. 화면엔 아직 없음

`plot 등`

→

STEP 3

꾸미기

제목·축 이름·범례 붙이기

`title · xlabel · legend`

→

STEP 4

보여주기

show를 만나야 그제서야 화면에 짠 하고 나타남

`show`

재료 손질 → 조리 → 접시에 담기 → 손님에게 내놓기 — 요리와 같은 순서

posco

4단계 흐름의 핵심 순서

그리기와 꾸미기를 다 끝낸 다음, 맨 마지막에 show 한 번

01 · 준비·그리기

데이터 담고 plot 호출

그릴 데이터를 리스트나 데이터프레임으로 준비하고 plot에 전달

02 · 꾸미기

제목·축·범례 붙이기

생략하면 그래프는 그려지나, 무엇을 그린 건지 알 수 없게 됨

03 · 보여주기

맨 마지막에 show 한 번

show를 중간에 쓰면 그 뒤 꾸민 내용이 반영되지 않음

선이든 막대든 산점도든 같은 흐름 — 그리기 명령만 바뀜

posco

plt.show()의 역할

이 줄을 만나야 그래프가 **화면에 나타남**

CODE

```
plt.plot([1,2,3], [10,20,15])
plt.show()
# show 줄에서 그래프가 뜸
```

> **show는 그래프 작업의 마침표**
>
> 제목 붙이기 전에 쓰면 제목 없는 그래프가 먼저 나옴 — 항상 맨 마지막 한 곳에만

파이썬은 머릿속에 그려두고 기다리다가 show를 만나는 순간 한꺼번에 띄움

posco

한글이 깨지는 이유

코드 오류가 아니라 글꼴 설정 문제

01 · 증상

기본 글꼴에
한글 모양이 없음

글꼴에 한글 자형이 없어 네모 상자나 물음표로 대체 표시

네모로 깨짐

02 · 원인

코드 오류가 아닌
글꼴 설정 문제

matplotlib은 전 세계 공용 도구라 영어 글꼴이 기본값

자기 코드 의심 금지

03 · 해결

한글 글꼴 지정
두세 줄로 끝

rcParams로 font.family를 한글 글꼴 이름으로 지정하면 바로 해결

파일 맨 위 준비 운동

한국에서 그래프 그리는 사람이라면 거의 100% 처음에 겪는 문제

posco

한글 폰트 설정 코드

앞으로 모든 그래프 코드 맨 위에 붙일 준비물

CODE

```
plt.rcParams['font.family']='Malgun Gothic'
# 맥은 'AppleGothic'
plt.rcParams['axes.unicode_minus']=False
```

unicode_minus를 False로 끄는 이유

한글 글꼴을 쓰면 음수의 마이너스 기호가 깨짐 — 설비 데이터의 "어제보다 -3도"에 필수

외우려 하지 말고 실습 파일 맨 위에 복사해서 붙여 쓰는 습관

운영체제별 한글 글꼴

본인 환경에 맞는 글꼴 이름을 그대로 쓰면 됨

운영체제	font.family 이름
윈도우	`'Malgun Gothic'`
맥	`'AppleGothic'`
구글 코랩	`'Malgun Gothic'`

> **윈도우·코랩은 Malgun Gothic, 맥은 AppleGothic** — 이 두 가지만 기억

posco

실습 1. 환경 준비와 첫 그래프

한글 폰트 설정 후 리스트로 첫 선 그래프 출력

목표

한글 폰트를 설정하고 리스트로 첫 선 그래프 출력

단계

· pyplot을 plt 별칭으로 불러오기

· rcParams로 한글 폰트를 설정해 글자 깨짐 방지

· 요일·온도 리스트를 plot으로 그리고 화면에 표시

예상 결과

오른쪽 위로 상승하는 선 그래프 한 장

posco

실습 1. 환경 준비와 첫 그래프 정답

한글 폰트 설정 후 리스트로 첫 선 그래프 출력

CODE · 실습 1

```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Malgun Gothic'   # 맥: 'AppleGothic'
요일 = [1, 2, 3, 4, 5]
온도 = [58, 60, 63, 61, 66]
plt.plot(요일, 온도)
plt.show()
```

posco

plt.plot() — 점을 선으로 잇기

x와 y를 같은 순서끼리 짝지어 점을 만들고, 그 점들을 순서대로 잇기

plt.plot(x, y)는 x와 y를 같은 순서끼리 짝지어 점으로 만들고,
그 점들을 순서대로 **선으로 이음**

선 그래프는 **시간처럼 이어지는 변화**를 볼 때 가장 잘 어울림

예시 — 점 세 개의 선

x = `[1,2,3]`, y = `[10,20,15]` → (1,10) (2,20) (3,15) 세 점
plot이 이 점들을 순서대로 선으로 이음

posco

plt.plot() 기본 코드

x와 y는 개수가 반드시 같아야 함

CODE

```
x = [1, 2, 3, 4, 5]
y = [62, 63, 65, 68, 72]
plt.plot(x, y)
plt.show()
```

요일 5개와 온도 5개를 plot에 넣고 show로 띄우면 **상승하는 꺾은선**이 나옴

개수가 다르면 짝이 안 맞아 오류 — 두 리스트 길이를 꼭 맞추기

posco

x를 생략하면 일어나는 일

y만 넣으면 x는 0부터 자동 번호

CODE

```
plt.plot([62, 63, 65, 68, 72])
plt.show()
# x축은 0,1,2,3,4 자동
```

왜 0부터인가

파이썬은 순서를 0부터 셈 — 리스트 첫 값을 꺼낼 때 0번이라 했던 것과 같음
가로축에 의미가 있으면 x를 직접, 측정 순서만 보면 충분할 땐 y만 넣어 빠른 확인용

값 5개면 x는 자동으로 0·1·2·3·4

posco

선 색 지정하기

color 옵션으로 의미를 담아 — 빨강은 위험, 파랑은 안정

plot에 **color** 옵션을 더하면 선 색을 바꿀 수 있음

색을 의미 있게 쓰면 **위험한 설비를 한눈에** 표시 가능

AVAILABLE COLOR NAMES

`'red'` · `'blue'` · `'green'` · `'orange'` · `'black'` · `'gray'`

색 이름은 반드시 따옴표로 감싸기

한 그래프에 색 너무 많으면 오히려 정신없음 — 강조할 것만 눈에 띄는 색

posco

선 색 지정 코드

데이터 다음에 color 옵션을 쉼표로 이어 붙이기

CODE

```
plt.plot([1,2,3,4,5], [62,63,65,68,72],
 color='red')
plt.show()
# 빨간색 상승 선 그래프
```

plot에 **color='red'**를 쉼표로 이어 붙이면 빨간 선이 그려짐

이어서 선 모양과 점 표시도 같이 바꾸기

posco

선 스타일과 마커 지정하기

옵션은 쉼표로 계속 이어 붙여 한꺼번에 적용

01 · linestyle

선 모양을 실선·점선으로

실선·점선을 정함. 대시 두 개면 점선 — 흑백 인쇄에서도 선이 구분됨

```
linestyle='--'
```

02 · marker

데이터 점의 표시를 동그라미·네모로

데이터 찍힌 위치에 동그라미·네모 등을 표시 — 어디가 실제 측정값인지 보여줌

```
marker='o'
```

동그라미 마커에 실선·점선 정도만 손에 익히고 나머진 필요할 때 검색

posco

자주 쓰는 스타일과 마커

color · linestyle · marker — 쉼표로 한꺼번에 적용

옵션	값	모양
linestyle	`'-'` (기본)	실선
linestyle	`'--'` (대시 두 개)	점선
marker	`'o'`	동그라미
marker	`'s'`	네모

color · linestyle · marker를 한꺼번에 적용 가능 — 이어서 여러 선 겹쳐 그리기

posco

여러 선을 겹쳐 그리기

plot을 여러 번 쓰면 한 그래프에 선이 겹쳐 — 설비끼리 비교 가능

plot을 여러 번 쓰면 **show 전까지 그린 모든 선이 한 그래프에 겹침**
색은 지정 안 하면 자동으로 다르게 골라줌

---

label로 각 선에 이름 붙이고 **legend**로 그 이름들을 범례로 표시

> **겹쳐 그릴 때 주의**
>
> 단위가 비슷한 것끼리 묶기 — 온도(60~80)와 가동시간(6~9)을 함께 그리면
> 가동시간 선이 바닥에 깔려 안 보임

posco

두 설비 비교 코드

plot 두 번 + 각각 label + legend 호출

CODE

```
plt.plot(x, a온도, label='A호기')
plt.plot(x, b온도, label='B호기')
plt.legend()
plt.show()
```

plot 두 번 → 각각 label → **legend** 호출 → show

legend 줄을 빠뜨리면 범례가 안 나옴 — 가장 흔한 실수

posco

데이터를 리스트로 준비하기

그래프 그리기는 데이터 준비 + plot 명령, 데이터 준비가 절반

직접 만든 **리스트**와 CSV에서 불러온 **데이터프레임 열** 모두 plot에 사용 가능

주의할 점 셋

숫자인지 확인

그릴 데이터가 문자열이 아닌 숫자인지 먼저 확인

결측치 미리 처리

앞서 배운 `isna` · `dropna`로 결측치 제거

날짜는 정렬

`sort_values`로 정렬해야 선이 제대로 그려짐

데이터만 잘 준비되면 그래프는 한두 줄

posco

데이터 출처 두 가지

직접 만든 리스트 · CSV에서 불러온 열 — 둘 다 plot에 그대로

CODE

```
# 1. 직접 만든 리스트
plt.plot([1,2,3], [62,63,65])

# 2. CSV에서 불러온 데이터프레임 열
df = pd.read_csv('설비센서_샘플.csv')
plt.plot(df['일자'], df['온도'])
plt.show()
```

위는 직접 만든 리스트, 아래는 CSV에서 불러온 열 — **둘 다 plot에 그대로**

이어서 초보자가 자주 걸리는 실수 정리

posco

선 그래프에서 자주 하는 실수

그래프가 안 보일 때 점검 순서 세 가지

01

show 줄이 있는가

가장 흔한 원인 — 마지막 줄에 show가 있는지 가장 먼저 확인

02

x와 y의 개수가 같은가

길이 오류가 나면 두 리스트 길이부터 비교 — 점은 짝이 맞아야 만들어짐

03

한글 글꼴 설정을 했는가

한글이 네모로 깨지면 파일 맨 위의 글꼴 설정 확인

추가 — 여러 장으로 쪼개지면 show를 중간에 여러 번 쓴 것

실수 점검표

증상 → 원인 → 점검 — 한 줄로 외워두기

증상	원인	점검
안 보임	show 누락	마지막 줄
길이 오류	x · y 개수 다름	길이 비교
한글 깨짐	글꼴 누락	맨 위 설정

> **오류는 혼내는 게 아니라 어디가 문제인지 알려주는 안내문** — 마지막 줄만 읽어도 충분

posco

실습 2. 선 그래프 직접 그리기

색·선 스타일·마커를 바꿔 가며 선 그래프 그리기

목표

색·선 스타일·마커를 바꿔 가며 선 그래프 그리기

단계

· 설비 데이터를 불러와 A호기 행만 고르기

· 일자와 온도로 기본 선 그래프 그리기

· color·linestyle·marker로 선을 꾸며 다시 그리기

예상 결과

기본→색→점선·마커 순으로 바뀌는 선 그래프

posco

실습 2. 선 그래프 직접 그리기 정답

색·선 스타일·마커를 바꿔 가며 선 그래프 그리기

CODE · 실습 2

```
df = pd.read_csv('설비센서_샘플.csv')
a = df[df['설비명'] == 'A호기']
plt.plot(a['일자'], a['온도'])                          # 기본
plt.plot(a['일자'], a['온도'], color='green')           # 색
plt.plot(a['일자'], a['온도'], color='red', linestyle='--', marker='o')
plt.show()
```

posco

실습 3. 센서값 추세 선 그래프

실데이터로 한 설비 추세와 두 설비 비교 그리기

목표

실데이터로 한 설비 추세와 두 설비 비교 그리기

단계

· C호기 행을 골라 마커를 붙인 온도 추세 그리기

· A호기와 C호기를 겹쳐 그리고 각 선에 이름표 달기

· legend로 범례를 표시해 어느 선이 어느 설비인지 구분

예상 결과

C호기 추세와 A·C호기 비교 그래프(범례 포함)

posco

실습 3. 센서값 추세 선 그래프 정답

실데이터로 한 설비 추세와 두 설비 비교 그리기

CODE · 실습 3

```
c = df[df['설비명'] == 'C호기']
plt.plot(c['일자'], c['온도'], marker='o')      # C호기 추세
plt.plot(a['일자'], a['온도'], label='A호기')
plt.plot(c['일자'], c['온도'], label='C호기')   # 겹쳐 비교
plt.legend()
plt.show()
```

posco

제목·축이 없는 그래프의 문제

제목·축·범례가 없는 그래프는 미완성

01 · 제목 없음

그린 사람도 며칠 뒤 모름

당장은 기억해도 며칠 지나면 그린 본인도 무슨 그래프인지 헷갈림

제목은 그래프의 얼굴

02 · 축 이름 없음

가로축이 날짜인지 순서인지 불명

축 이름 없으면 가로축 숫자가 날짜인지 순서인지 알 수 없음

두 축의 의미를 못 잡음

03 · 범례 없음

선이 여럿이면 구분 불가

어느 선이 어떤 설비인지 알 수 없어 비교의 의미가 사라짐

여러 선 = 무조건 범례

이 세 가지를 다는 것 — 이 모듈의 학습 목표

posco

plt.title()로 제목 달기

그래프 위쪽 가운데에 제목 표시 — 명령 하나로 끝

plt.title('제목')을 추가하면 그래프 위쪽 가운데에 제목 표시

좋은 제목은 **설비명·값·기준**이 드러나게 작성

❌ 나쁜 제목

온도

어떤 설비? 어떤 기준?

☑ 좋은 제목

C호기 일자별 온도 변화

설비명 + 값 + 기준 모두 명확

posco

제목 달기 코드

선을 그린 뒤 title 한 줄 추가 — show 앞에

CODE

```
plt.plot([1,2,3,4,5], [75,76,78,80,83])
plt.title('C호기 일자별 온도 변화')
plt.show()
```

상단 가운데에 한글 제목이 붙음 — 제목은 보는 사람을 위한 안내

축 이름 달기 (xlabel·ylabel)

축 이름에는 단위까지 함께 적어야 의미가 분명

01 · xlabel

가로축 이름

x는 가로, label은 이름표

02 · ylabel

세로축 이름

y는 세로, label은 이름표

단위가 없으면 70이 70도인지 70%인지 알 수 없음

온도 → **온도(℃)** · 진동 → **진동(mm/s)** · 가동시간 → **가동시간(시간)**
제목·가로축 이름·세로축 이름은 항상 한 세트로 함께

축 이름 없으면 가로축 숫자가 날짜인지 순서인지 알 수 없음

posco

축 이름 달기 코드

제목 + 두 축 이름까지 — 기본 설명 세트

CODE

```
plt.plot([1,2,3,4,5], [75,76,78,80,83])
plt.title('C호기 일자별 온도 변화')
plt.xlabel('일자')
plt.ylabel('온도(℃)')
plt.show()
```

제목·두 축까지 붙이면 누가 봐도 이해 가능 — 이어서 가로축 글자 바꾸기

posco

축 눈금 바꾸기 (xticks)

데이터는 그대로 두고 보여주는 방식만 글자로 교체

plt.xticks(위치, 글자)로 가로축 눈금에 표시되는 값을 직접 교체

위치 리스트와 글자 리스트는 **순서대로 짝**, 길이가 같아야 함

데이터는 그대로, 표시만 바꾸기

요일이 1·2·3 숫자로 저장돼 있어도 그래프엔 **월·화·수** 글자가 직관적
긴 글자는 `rotation` 옵션으로 기울여 표시

plot의 x·y 규칙처럼 두 리스트 길이 일치 필수

posco

축 눈금 바꾸기 코드

가로축 1~5 자리에 월·화·수·목·금 표시

CODE

```
plt.plot([1,2,3,4,5], [62,63,65,68,72])
plt.xticks([1,2,3,4,5],
 ['월','화','수','목','금'])
plt.show()
```

가로축 글자를 원하는 대로 — 이어서 범례 제대로 다루기

posco

plt.legend()와 label의 연결

label은 이름 적어두기, legend는 그 이름 보여주기 — 반드시 짝으로

01 · label

이름
적어두기

plot에 label로 각 선의 이름을 적어둠 — 이것만으론 범례가 안 나옴

```
plt.plot(…, label='A호기')
```

02 · legend

이름
보여주기

plt.legend()를 호출해야 label로 적어둔 이름들이 범례로 나타남

```
plt.legend()
```

legend를 빠뜨리면 label 붙여도 범례가 안 나옴 — 가장 흔한 실수

posco

범례 코드

각 plot에 label → 맨 마지막에 legend 한 번

CODE

```
plt.plot(x, a, label='A호기')
plt.plot(x, b, label='B호기')
plt.legend()
plt.show()
```

label은 선을 그리는 **plot 안에**, legend는 맨 마지막 한 번

범례가 선과 겹쳐 안 보일 때 위치를 옮기는 법으로 이어짐

posco

범례 위치 정하기 (loc)

데이터가 비어 있는 쪽으로 범례를 보내는 것이 요령

plt.legend(loc='위치')로 범례를 원하는 모서리로 이동

---

loc는 **location**의 줄임말 — upper·lower + left·right 조합

빈 공간으로 보내는 요령

상승하는 데이터는 보통 **왼쪽 위**가 비어 있어 그 자리에 두면 안 겹침
loc를 지정하지 않으면 `best` 기본값 — matplotlib이 알아서 덜 가리는 곳을 선택

자동 배치가 마음에 안 들 때만 직접 옮김

범례 위치 값

upper·lower + left·right의 네 가지 조합

loc 값	위치
`'upper left'`	왼쪽 위
`'upper right'`	오른쪽 위
`'lower left'`	왼쪽 아래
`'lower right'`	오른쪽 아래

외울 필요 없이 빈 공간을 보고 그때그때 선택

posco

figsize로 그래프 크기 조절하기

도화지 크기를 먼저 잡고 그 위에 그리는 순서 — figure는 plot보다 먼저

plt.figure(figsize=(가로, 세로))로 그래프 전체 크기를 인치로 지정

---

figure는 반드시 **plot보다 먼저** 작성

데이터 많으면 크게, 적으면 작게

시간에 따라 길게 이어지는 데이터는 **가로를 길게**(예: 10, 4) 잡으면 추세가 잘 보임
plot 다음에 figure를 쓰면 적용되지 않음 — 순서 주의

빽빽해서 안 보이면 가로를 늘리기

posco

figsize 코드

figure를 plot보다 먼저 작성 — 가로로 긴 그래프

CODE

```
plt.figure(figsize=(10, 4))
plt.plot([1,2,3,4,5,6,7],
 [62,63,65,68,72,75,78])
plt.show()
```

크기가 안 먹힌다는 질문은 대부분 **순서가 뒤바뀐 경우**

이어서 배경 격자 표시

posco

격자 표시하기 (grid)

격자는 주인공이 아닌 **값 읽기를 돕는 보조선**

01 · plt.grid(True)

배경에 격자선

가로세로 격자선이 생겨 값 읽기 쉬움 — 70과 75 사이면 72쯤이라고 빠빠르게 읽힘

```
plt.grid(True)
```

02 · alpha

흐린 격자로 은은하게

격자가 진하면 데이터보다 튐 — alpha를 0.3쯤으로 낮춰 보조선 역할만

```
plt.grid(True, alpha=0.3)
```

점검 기준 75도라면, 75 격자선을 따라 눈을 옮겨 언제 넘었는지 빠르게 확인

posco

그래프를 이미지 파일로 저장하기 (savefig)

savefig는 반드시 show보다 먼저 작성

plt.savefig('파일명.png')로 그래프를 이미지 파일로 저장

그리기 → 꾸미기 → **savefig** → show 순서가 안전

왜 show 다음에 저장하면 안 되는가

show로 화면에 띄우면 matplotlib이 그래프를 비워버려, 그 다음 savefig하면 **빈 그림이 저장**
더 선명하게 — `dpi=150` 또는 `dpi=200`으로 높이기

파일은 작업 폴더에 생성 · os.path.exists로 확인 가능

posco

저장 코드

savefig를 show보다 먼저 — 작업 폴더에 이미지 저장

CODE

```
plt.plot([1,2,3], [62,65,72])
plt.title('C호기 온도 변화')
plt.savefig('C호기_온도.png')
plt.show()
```

분석 결과를 **두고두고 다시 보고 공유**할 수 있게 남기기

이어서 정확한 색을 위한 16진수 색상 코드

색 이름과 16진수 색상 코드

색상 코드는 외우지 말고 color picker에서 골라 복사

'red' 같은 색 이름 외에 **#RRGGBB** 형식의 16진수 코드로 정확한 색 지정 가능

앞 두 자리 빨강, 가운데 초록, 뒤 파랑의 세기 — 화면의 **모든 색**은 이 셋의 조합

노베이스 단계에선 기본 색 이름으로 충분

더 정밀한 색이 필요하면 #FF5733 같은 우물정자 코드 사용

color picker에서 원하는 색을 골라 코드를 그대로 복사해 쓰면 됨

보고서 그래프 색을 통일하면 전문적으로 보임

posco

색 지정 두 가지 방식

색 이름 vs 16진수 — 상황에 맞게

방식	예시	특징
색 이름	`'red', 'blue'`	쉽지만 한정
16진수	`'#FF5733'` 형식	자유 지정

보고서 그래프 색을 통일하면 전문적으로 보임 — 같은 16진수를 모든 그래프에 재사용

posco

코드 작성 순서가 결과에 미치는 영향

figure 맨 앞 · show 맨 뒤 · savefig는 show 바로 앞

figure로 크기 → plot으로 그리기 → 꾸미기 → savefig로 저장 → 맨 마지막 **show**

그 사이 꾸미기 명령들끼리는 **순서를 바꿔도 결과가 같음**

펴고 → 그리고 → 달고 → 찍고 → 보여준다

일상의 그림 그리는 순서와 같음 — 다섯 동작이 곧 코드 순서

title·xlabel·ylabel·legend는 각자 다른 부분을 건드려 순서 무관

세 위치만 지키면 결과는 어긋나지 않음

posco

올바른 코드 순서

figure → plot → 꾸미기 → savefig → show

STEP 1

크기

도화지를 먼저 잡기. figsize로 가로·세로 인치 지정

`plt.figure`

→

STEP 2

그리기

plot으로 데이터를 선으로 바꾸기. 옵션은 쉼표로 이어 붙임

`plt.plot`

→

STEP 3

제목·축

title·xlabel·ylabel·legend로 꾸미기. 이 안은 순서 무관

`title · xlabel · legend`

→

STEP 4

저장

savefig로 이미지 파일 저장. 반드시 show 바로 앞

`plt.savefig`

→

STEP 5

보여주기

show로 화면 표시. 다섯 단계의 마침표

`plt.show`

실습을 반복하면 이 순서가 손에 배어 의식 안 해도 자동으로 쓰게 됨

읽을 수 있는 그래프의 조건

그래프를 그린 뒤 네 가지 점검 체크리스트

점검 항목	확인 내용
제목	무슨 그래프인지 한눈에 알 수 있는가
축 이름	두 축의 의미와 단위가 적혀 있는가
범례	선이 여럿일 때 어느 선이 어떤 설비인지 구분되는가
크기	빽빽하거나 너무 작아 안 보이는 곳이 없는가

네 가지가 갖춰지면 **설명 없이도 혼자 이해됨** — 그게 읽을 수 있는 그래프

그래프 완성도의 핵심

화려함보다 기본을 빠뜨리지 않는 것이 중요

01 · 절대 기본

제목과 축 이름은 빠뜨리면 안 됨

단위가 명확한가, 색이 의미 있게 쓰였는가, 데이터가 정렬됐는가까지 점검

기본만 확실해도 충분

02 · 신뢰의 갈림길

마지막 꾸미기에서 신뢰도가 갈림

같은 데이터로 그려도 제목·축·범례가 빠지면 의심받음 — 분석이 아무리 좋아도 인정 어려움

엉성한 그래프는 의심 유발

03 · 핵심 가치

화려함보다 기본을 챙기는 일

완벽하게 꾸미려 욕심내지 않기 — 제목·두 축·범례면 충분

충분히 읽을 만한 그래프

이어서 모든 걸 한 그래프에 적용하는 완성형 실습으로

posco

실습 4. 제목·축·범례 갖춘 완성형 그래프

제목·축 이름·범례를 모두 갖춘 읽히는 그래프

목표

제목·축 이름·범례를 모두 갖춘 읽히는 그래프 완성

단계

· figsize로 가로로 긴 그래프 틀 만들기(plot보다 먼저)

· 두 설비를 색·마커를 달리해 겹쳐 그리기

· 제목·가로축·세로축 이름과 범례 위치를 지정

예상 결과

제목·축이름·범례를 갖춘 두 설비 비교 그래프

posco

실습 4. 제목·축·범례 갖춘 완성형 그래프 정답

제목·축 이름·범례를 모두 갖춘 읽히는 그래프

CODE · 실습 4

```
plt.figure(figsize=(10, 4))
plt.plot(a['일자'], a['온도'], label='A호기', color='blue', marker='o')
plt.plot(c['일자'], c['온도'], label='C호기', color='red', marker='s')
plt.title('A호기 vs C호기 온도 변화')
plt.xlabel('일자'); plt.ylabel('온도(℃)')
plt.legend(loc='upper left')
plt.show()
```

posco

실습 5. 다중 센서 비교 그래프와 저장

반복문으로 세 설비를 비교하고 이미지로 저장

목표

반복문으로 세 설비를 겹쳐 비교하고 이미지로 저장

단계

· 설비 이름과 색 목록을 반복하며 각 설비 온도 선 그리기

· 제목·축 이름·범례로 그래프를 완성

· savefig로 그래프를 이미지 파일로 저장(show보다 먼저)

예상 결과

세 설비 온도 비교 그래프가 파일로 저장

posco

실습 5. 다중 센서 비교 그래프와 저장 정답

반복문으로 세 설비를 비교하고 이미지로 저장

CODE · 실습 5

```
plt.figure(figsize=(10, 4))
for name, color in [('A호기','blue'), ('B호기','green'), ('C호기','red')]:
    sub = df[df['설비명'] == name]
    plt.plot(sub['일자'], sub['온도'], label=name, color=color, marker='o')
plt.title('설비별 온도 비교')
plt.xlabel('일자'); plt.ylabel('온도(℃)')
plt.legend()
plt.savefig('설비별_온도.png')
```

posco

실습 6. 선 그래프 자유 변형

한 설비의 여러 센서를 한 그래프에 종합

목표

한 설비의 여러 센서를 한 그래프에 종합해 그리기

단계

· C호기의 온도·진동·소음을 한 그래프에 겹쳐 그리기

· 단위가 다른 진동은 배수를 곱해 눈에 보이게 보정

· 제목·축 이름·범례로 완성하고 함께 움직이는지 관찰

예상 결과

C호기 온도·진동·소음이 함께 상승하는 종합 그래프

posco

실습 6. 선 그래프 자유 변형 정답

한 설비의 여러 센서를 한 그래프에 종합

CODE · 실습 6

```
plt.figure(figsize=(10, 4))
plt.plot(c['일자'], c['온도'], label='온도', marker='o')
plt.plot(c['일자'], c['진동'] * 20, label='진동(20배)', marker='s')
plt.plot(c['일자'], c['소음'], label='소음', marker='^')
plt.title('C호기 센서 종합')
plt.xlabel('일자'); plt.ylabel('측정값')
plt.legend()
plt.show()
```